

Arithmetic Intensity $\uparrow = T_{math} > T_{comm} = \text{use most flops} = \text{compute-bound}$
 Arithmetic Intensity $\downarrow = T_{math} < T_{comm} = \text{flops wasted} = \text{communication bound}$

peak arithmetic intensity (accelerator) = peak FLOPs/s e.g. TPU v2e MXU $\frac{\text{Accelerator FLOPs/s}}{\text{Bandwidth Bytes/s}} = \frac{1.97 \text{E}19}{8.2 \text{E}11} = 240 \text{ FLOPs/byte}$

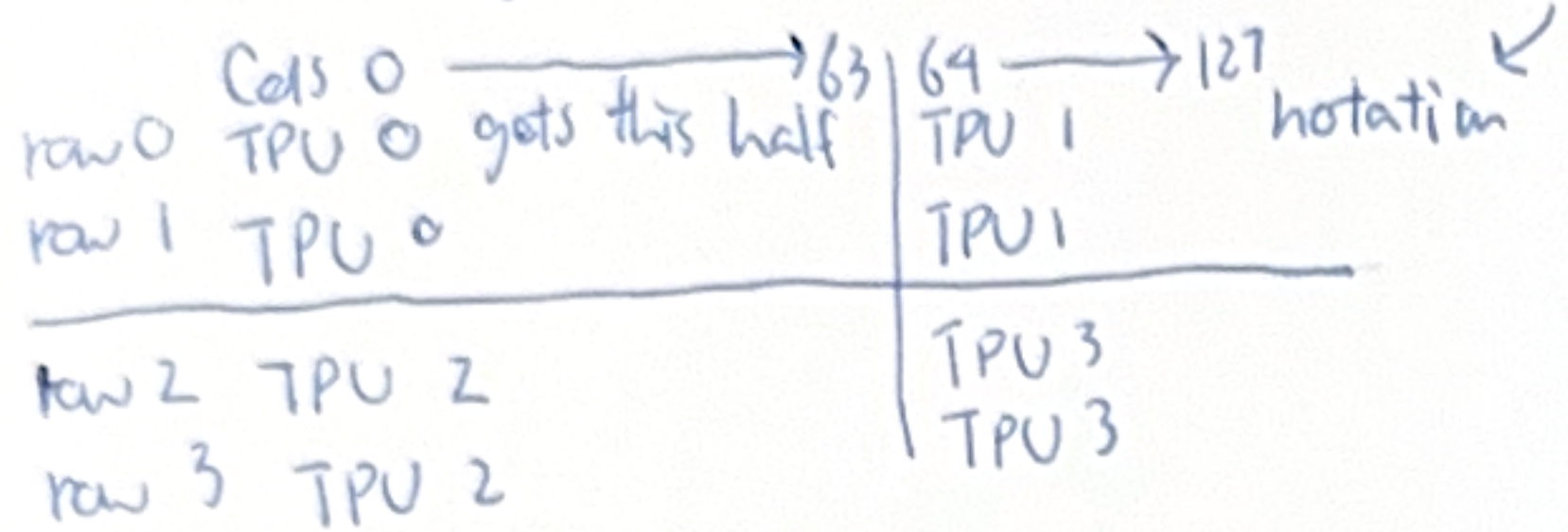
ratio can be > 1
 improve performance of algorithm by increasing arithmetic intensity / memory bandwidth available

TPU [8, 128] Each position has 4 ALU
 FLOPs/cycle $8 \times 128 \times 4 = 4096$

MXU (128, 128) systolic array [256, 256] on TPU v4e
 Each position has 1 ALU

The art of scaling is figuring out how to shard our model so computation remains efficient.

4 TPUs arranged in 2x2 grid (mesh: {X:2, Y:2})



Nothing is duplicated
 global shape: (4, 128) local shape: (2, 64)

Mesh (devices = ((0,1), (2,3)), axis-handles = ('x', 'y'))

outer type: 2 items inside = 2 rows x mesh axis
 inner type: 2 items inside = 2 columns y mesh axis

2x2 grid immutable list

Array A[1, J]

Sharding A[1x, Jx] shard the first axis I, along the mesh axis X, and the second axis J, along the mesh axis Y.

this tells us that each shard holds $\frac{1}{|X| \cdot |Y|}$ e.g. $\frac{1}{2 \cdot 2} = \frac{1}{4}$

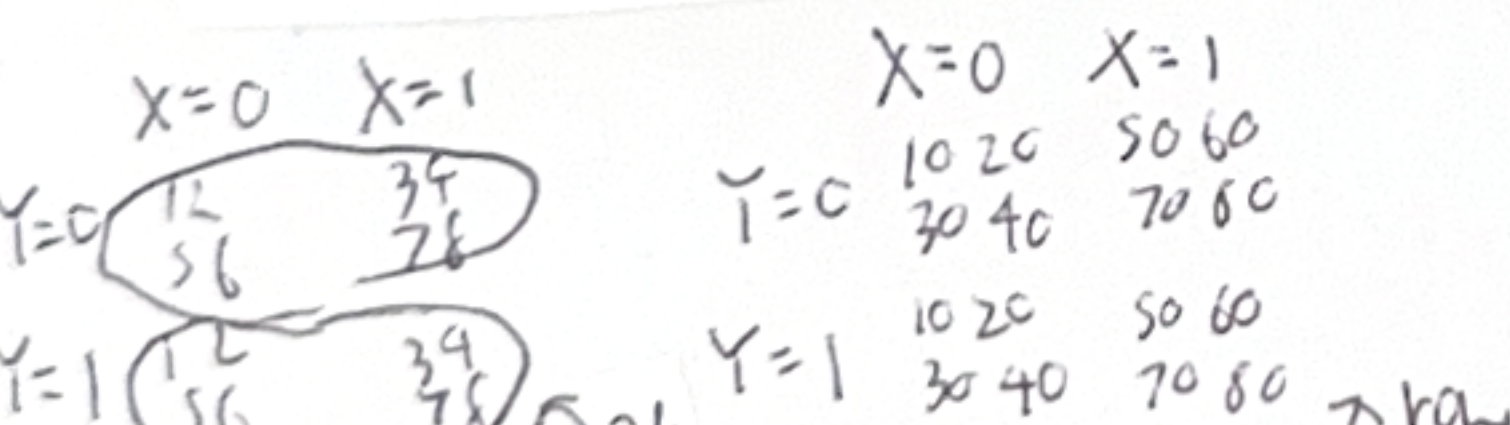
Size of shard ie local shape (|I|/2, |J|/2) mesh axis

size of first dim 4 128

$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{bmatrix}$ $B = \begin{bmatrix} 10 & 20 \\ 30 & 40 \\ 50 & 60 \\ 70 & 80 \end{bmatrix}$

$A[1, Jx]$ $B[Jx, K]$

Different X coordinates have different items



The sharding depends on the dimension

Along Y is replicated because AB isn't sharded along Y axis

AllGather_x [Bx, Dy] $\rightarrow$ [B, Dy]

data moved = $\frac{BD}{1 \text{ or the sharded dimension (if still sharded)}}$

$\frac{1}{4}$ of the whole array

AllGather = bytes transferred / WxHxK, axis-handles

Heads (1 separate copy of the Q/K/V but with its own weights)

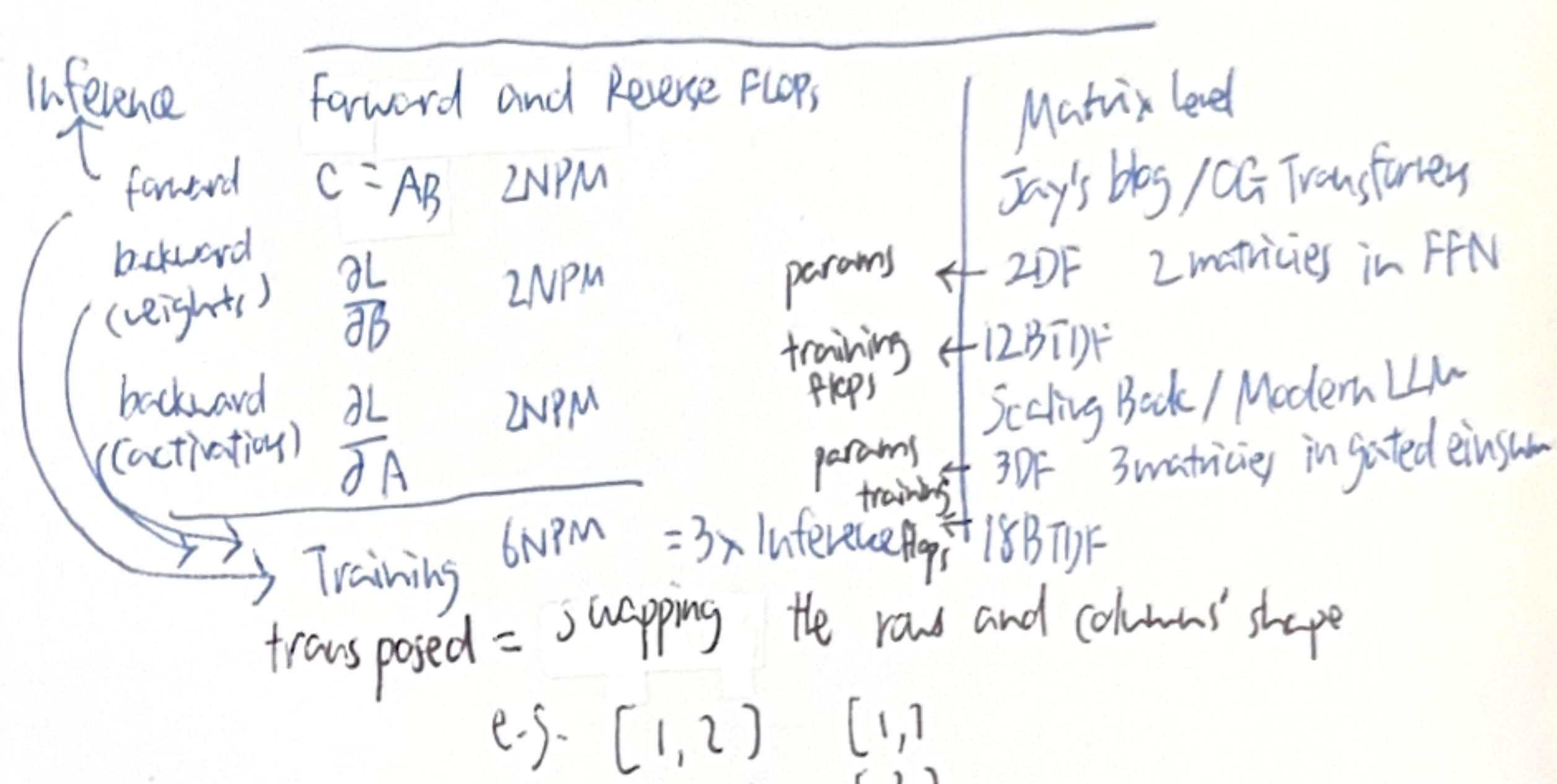
MHA (Multi-head attention) N = query viewpoints N=K e.g. 8 query heads K = KV viewpoints 8 key and value heads

MQA (Multi-query attention) N = a large number K=1

GQA (Grouped-query attention) N=8 K=2 Query 0-3 share KV 0 Query 4-7 share KV 1 G = 8/2 = 4 query heads/KV head

6 x params x hidden tokens = training flops

6 x params = each token cost



Section 4 x [P] is 2D-dimensional vector its shape is just [P] x [P, 1] or [1, P]

FSPP D/F (model dimensions) are sharded but ~~not~~ before matmul. B is sharded
 Tensor parallelism D/F are sharded

T < 8D attention is small fraction of total training flops

T = 8D attention costs about the same as all projection + MLP

T > 8D attention majority of flops in training

Large data = $T_{tr} = \frac{V}{W_{bidi}}$

Small data = $T_{trp} = \frac{\text{data per device}}{W_{mini}}$

Element-wise operations 2 tensors/matrices of same shape to multiply on matching positions

Dot product multiply + sum vectors of the same length $\rightarrow$ 7D arrays output is a scalar

Matmul (many dot products) C = AB

